United States Capital City Picker

Project 3

Alec Krsek
aleckrsek@oregonstate.edu

October 18th, 2026

Oregon State University - Cascades

Introduction

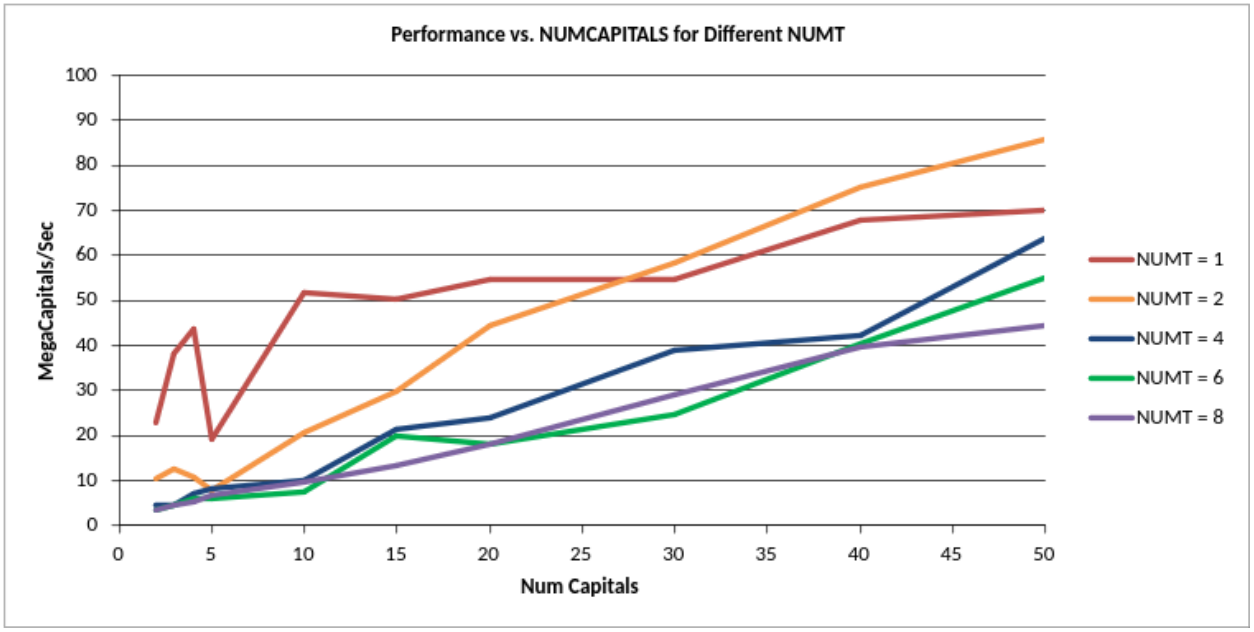
In this project, we use multiple threads to speed up our capital picking algorithm. The algorithm divides the cities in our input data into X number of regions. Our bash script has a range of regions that is selected for each run. Each iteration computes the distance between each capital and sums it. At the end, it takes the average of the longitude and latitude. This average is used to find the closest city and mark that as a capital. This process is called the k-means algorithm.

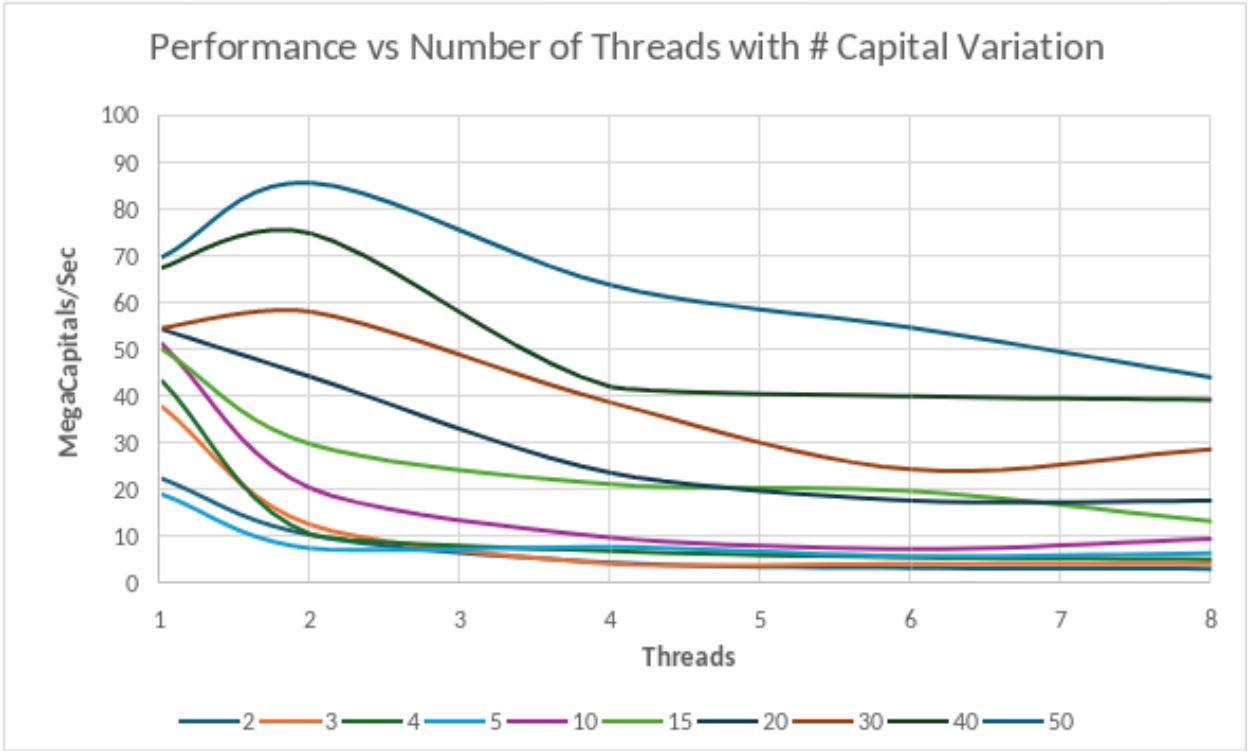
Workstation Specifications

Operating System: Fedora Linux (Kernel 7.2.9) on bare metal
CPU Name: I5-1335U
Cores: 10
Threads per core: 2
RAM: 24 GB
Cache:
- L1d: 352 KiB
- L1i: 576 KiB
- L2: 6.5 MiB
- L3: 12 MiB

Compiler: GCC

Results





In the previous projects, we usually see a pattern that with more threads we usually get more performance. But when looking at the first graph, we can see that having one thread gives us the most performance until we pass 25 capitals. After that, 2 threads is the best performing. Using four threads and more throughout the entire experiment never gave us better performance than using 1 or 2 threads. My guess is that there may be false sharing or threads waiting for each other to write to some shared data. We also see that the more capitals there are in a run, the better performance we get. Running 2

threads at 30 capitals or more gives us better performance than one thread because we have enough work to benefit from parallelism.

The bottom graph gives us the same information on the data trends, but displays it in another way. This bottom graph shows us that 2 threads is the best performing for 3 sets of series: 30, 40 and 50. The rest of the series data shows that 1 thread is the best performing, then crashing down at 2 and more threads.

NumT	Capitals	MegaCityCapitals/ Sec
1	2	22.653
1	3	38.170
1	4	43.695
1	5	19.308
1	10	51.602
1	15	50.416
1	20	54.598
1	30	54.779
1	40	67.683
1	50	69.858
2	2	10.532
2	3	12.651
2	4	10.641
2	5	7.641
2	10	20.480
2	15	29.930
2	20	44.355

2	30	58.285
2	40	75.039
2	50	85.846
4	2	4.620
4	3	4.430
4	4	7.139
4	5	7.990
4	10	9.999
4	15	21.380
4	20	23.781
4	30	38.836
4	40	42.177
4	50	63.921
6	2	3.532
6	3	4.416
6	4	5.786
6	5	6.029
6	10	7.550
6	15	19.944
6	20	17.916
6	30	24.548
6	40	40.252
6	50	54.866
8	2	3.311

8	3	4.359
8	4	5.363
8	5	6.657
8	10	9.687
8	15	13.431
8	20	17.912
8	30	28.907
8	40	39.454
8	50	44.268